

Arrays & Functions in C

Creating and manipulating arrays • Passing arrays to functions

Menna | Technical Interview Demo | July 19, 2026

BEFORE WE START

Assumed Student Background

1. **Comfortable with variables & data types**

   int, float, char

2. **Comfortable with loops**

   for, while

3. **Comfortable with conditionals**

   if / else

4. **NEW today: arrays and functions with arrays**

   first exposure

If any assumption doesn't hold for this room, I'll adjust on the fly.

MOTIVATION

Why Do We Need Arrays?

Without arrays

```
int grade1 = 90;
int grade2 = 85;
int grade3 = 77;
int grade4 = 60;
int grade5 = 95;
// ...for 100 students??
```

With one array

```
int grades[5] =
    {90, 85, 77, 60, 95};

// one name, many values
// scales to any size
```

Key idea: An array is a collection of same-type elements stored sequentially (back-to-back) in memory, under a single name.

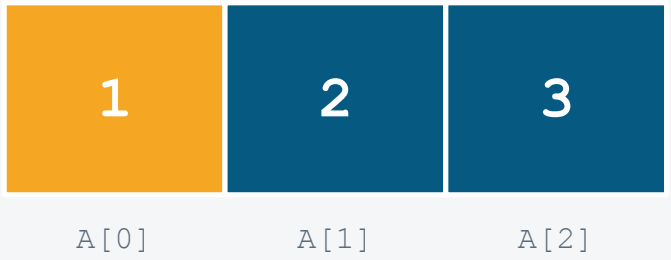
ARRAYS IN C

Declaring & Accessing Arrays

```
int A[] = {1, 2, 3};

printf("%d", A[0]);
  // 1
printf("%d", A[1]);
  // 2
printf("%d", A[2]);
  // 3
printf("%d", A[3]);
  // garbage - undefined behavior
```

Memory layout (contiguous)



base address + (index × size) → any element's location

⚠ **C does not check bounds for you.** Accessing A[3] on a 3-element array is undefined behavior — it might print garbage, crash, or silently corrupt other data.

Looping Without Knowing the Size

Fixed loop (you know the size)

```
for (int i = 0; i < 3; i++) {
    printf("%d", A[i]);
}
```

What if the size is unknown?

```
sizeof(A[0])
  // 4 (bytes per int)
sizeof(A)
     // 12 (3 × 4 bytes)

int size = sizeof(A) / sizeof(A[0]);
```

Dynamic loop

```
int size = sizeof(A) / sizeof(A[0]);

for (int i = 0; i < size; i++) {
    printf("%d ", A[i]);
}
```

This trick only works locally.

Once an array is passed into a function, this same sizeof() call gives the wrong answer — more on that shortly.

Strings: A Special Case of Char Arrays

```
char name[] = "menna";

printf("%c", name[0]);
  // m
printf("%s", name);
     // menna
```

`%c` prints ONE character.

`%s` prints characters until it hits a hidden **null terminator** ('\0') that C adds automatically at the end of the string.



Ordinary int arrays have no terminator — that's why they always need a size passed alongside them.

Arrays lose their size when passed to a function.

That's the whole reason the next part matters.

PASSING ARRAYS TO FUNCTIONS

Functions: The Biscuit Machine



Input goes in → the function processes it → output comes out.

```
output_type function_name(input_type arg) {
    // code
    return output;
}
```

Two behaviors depending on what you pass in: pass-by-value vs. pass-by-reference

PASSING ARRAYS TO FUNCTIONS

Pass-by-Value vs. Arrays

int → pass by value

```
int x = 2;

void add2(int n) {
    n = n + 2;
}

add2(x);
printf("%d", x);
 // 2 (unchanged!)
```

array → pass by reference

```
int arr[] = {1,2,3,4,5};

void add2(int n[]) {
    n[0] = n[0] + 2;
}

add2(arr);
printf("%d", arr[0]);
 // 3 (changed!)
```

C cannot copy a whole array into a function call. So arrays always behave like reference passing — whether you want that or not.

Proof: Same Address, No Copy

```c
int arr[] = {1, 2, 3, 4, 5};
printf("%p", arr);
        // address A

void add2(int n[]) {
    printf("%p", n);
    // same address A
}
```

Arrays decay to a pointer.

The array's name becomes the address of its first element when passed. No data is copied — just that address.

`int n[]` and `int *n`

are identical as function parameters. This is also why the size must be passed separately — the address alone can't tell you how many elements follow.

Recap

1. **Declare** — `int A[] = {1,2,3};`
2. **Access** — `A[i]` — 0-indexed, no bounds check
3. **Loop** — known size, or `sizeof(A)/sizeof(A[0])`
4. **Pass to a function** — always behaves like reference — pass size too

Questions?